

Flask-SQLAlchemy Validations

 [_ \(https://github.com/learn-co-curriculum/python-p4-validation-flask-sqlalchemy-validations\)](https://github.com/learn-co-curriculum/python-p4-validation-flask-sqlalchemy-validations)  [_ \(https://github.com/learn-co-curriculum/python-p4-validation-flask-sqlalchemy-validations/issues/new\)](https://github.com/learn-co-curriculum/python-p4-validation-flask-sqlalchemy-validations/issues/new)

Learning Goals

- Define constraints and validations in data processing.
- Ensure that only acceptable input is sent to the database using validations.

Key Vocab

- **Constraint:** a rule enforced on the data columns of a table. Ensures that only appropriate data is saved to the database.
- **Validation:** an automatic check to ensure that data entered is sensible and feasible.
- **Forms:** A web form (or HTML form) is a place where users enter data or personal information that's then sent to a server for processing.

Context: Databases and Data Validity

What is a "validation"?

In the context of Python, **validations** are special method calls that go at the top of model class definitions and prevent them from being saved to the database if their data doesn't look right.

In general, **validations** consist of code that performs the job of protecting the database from invalid data.

SQLAlchemy can validate our models for us before they even touch the database. This means it's harder to end up with bad data, which can cause problems later even if our code is technically bug-free.

We can use `SQLAlchemy` helper methods like `validates()` to set things up.

SQLAlchemy Validations vs Database Constraints

Many relational databases, such as SQLite and PostgreSQL, have data validation features that check things like length and data type. While SQLAlchemy validations are added in the model files, these validations are typically added via migrations.

Database constraints and model validations are also functionally different. Database constraints will ALWAYS be checked when adding or updating data in the database, while SQLAlchemy validations will only be checked when adding or updating data through the SQLAlchemy ORM (e.g. if we use SQL code in the command line to modify the database, SQLAlchemy validations are not run).

Some developers use both database constraints and SQLAlchemy Validations, while others rely on SQLAlchemy Validations alone. Ultimately, it depends on how the developer plans to add and update data in the database. In this lesson, we'll be focusing on SQLAlchemy Validations.

What is "Invalid Data"?

Suppose you get a new phone and you ask all of your friends for their phone number again. One of them tells you, "555-868-902". If you're paying attention, you'll probably wrinkle your nose and think, "Wait a minute. That doesn't sound like a real phone number."

"555-868-902" is an example of **invalid data**... for a phone number. It's probably a valid account number for some internet service provider in Alaska, but there's no way to figure out what your friend's phone number is from those nine numbers. It's a showstopper, and even worse, it kind of looks like valid data if you're not looking closely.

Validations Protect the Database

Invalid data is the bogeyman of web applications: it hides in your database until the worst possible moment, then jumps out and ruins everything by causing confusing errors.

Imagine the phone number above being saved to the database in an application that makes automatic calls using the Twilio API. When your system tries to call this number, there will be an error because no such phone number exists, which means you need to have an entire branch of code dedicated to handling *just* that edge case.

It would be much easier if you never have bad data in the first place, so you can focus on handling edge cases that are truly unpredictable.

That's where validations come in.

Basic Usage

For more examples of basic validation usage, see the SQLAlchemy Guide for [SQLAlchemy Validations](https://docs.sqlalchemy.org/en/14/orm/mapped_attributes.html)  (https://docs.sqlalchemy.org/en/14/orm/mapped_attributes.html).

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import validates
db = SQLAlchemy()

class EmailAddress(db.Model):
    __tablename__ = 'emailaddress'

    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String)
    backup_email = db.Column(db.String)

    @validates('email')
    def validate_email(self, key, address):
        if '@' not in address:
            raise ValueError("Failed simple email validation")
        return address
```

If we create an EmailAddress object we should expect a ValueError exception.

```
email = EmailAddress(email='banana')
session.add(email)
# => ValueError: Failed simple email validation
```

In this example, we wrote a `validate_email()` function, preventing the object from being saved if its `email` attribute does not include `@`. We can return a custom message by raising a `ValueError` with the message.

`validates` is our Swiss Army knife for validations. First you need to use a decorator which takes a string of the columns you want to validate. the first argument is the **key** we want to validate (the key's value will be the 'email'), and the second argument is the value of what we want to validate. We can validate multiple columns if we pass multiple column names into the `validates` decorator.

Here is an example of validating multiple columns with one validate function.

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import validates
db = SQLAlchemy()

class EmailAddress(db.Model):
    __tablename__ = 'emailaddress'

    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String)
    backup_email = db.Column(db.String)

    @validates('email', 'backup_email')
    def validate_email(self, key, address):
        if '@' not in address:
            raise ValueError("Failed simple email validation")
        return address
```

In this example the `validate_email` function will validate both `email` and the `backup_email` column. We can figure out which column we are validating by checking the `key` attribute. The key attribute will be `email` or `backup_email` because those are the columns we passed into the decorator.

Conclusion

In this lesson, we learned the importance of validating data to ensure that no bad data ends up in our database. We also discussed the difference between model validations and database constraints. Finally, we saw some common methods for implementing validations on our models using SQLAlchemy.

Solution Code

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import validates
db = SQLAlchemy()

class EmailAddress(db.Model):
    __tablename__ = 'emailaddress'

    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String)
    backup_email = db.Column(db.String)

    @validates('email', 'backup_email')
    def validate_email(self, key, address):
        if '@' not in address:
            raise ValueError("Failed simple email validation")
        return address
```

Resources

- [Changing Attribute Behavior - SQLAlchemy](https://docs.sqlalchemy.org/en/14/orm/mapped_attributes.html)  [_ \(https://docs.sqlalchemy.org/en/14/orm/mapped_attributes.html\)](https://docs.sqlalchemy.org/en/14/orm/mapped_attributes.html)